TP n°2 – POO PHP et base de données

PDO, objets métier, Repository et mapping objet-relationnel manuel

BTS SIO – Option SLAM – 2e année

Objectifs

Ce TP permet de mettre en pratique l'accès à une base de données dans une application PHP orientée objet.

À la fin du TP, vous devrez être capable de :

- utiliser PDO pour communiquer avec MySQL ;
- exécuter des requêtes préparées ;
- représenter une ligne de base de données par un objet PHP ;
- réaliser une hydratation manuelle ;
- créer un Repository ;
- mettre en œuvre les opérations CRUD ;
- séparer les données métier de l'accès à la base ;
- écrire une architecture simple proche du fonctionnement d'un ORM.

Base de données utilisée

Tous les exercices utilisent une base nommée :

`boutique`

Créez la table suivante :

```
CREATE TABLE produit (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prix DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);
```

Ajoutez quelques données :

```
INSERT INTO produit(nom, prix, stock) VALUES
('Clavier mecanique', 79.90, 10),
('Souris USB', 24.90, 25),
('Ecran 24 pouces', 159.90, 8),
('Casque audio', 49.90, 15),
('Webcam', 39.90, 12);
```

Connexion PDO

Vous pouvez utiliser le fichier suivant :

```
<?php

$host = "localhost";
$port = 3306;
$dbname = "boutique";
$user = "root";
$password = "";

try {

    $pdo = new PDO(
        "mysql:host=$host;port=$port;dbname=$dbname;charset=utf8mb4",
        $user,
        $password
    );

    $pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );

} catch (PDOException $e) {

    die(
        "Erreur de connexion : "
        . $e->getMessage()
    );
}
```

Adaptez éventuellement le port, l'utilisateur et le mot de passe à votre environnement.

1 Exercice 1 – Lire une ligne avec PDO

Difficulté : ⋆

Dans un fichier `exercice1.php`, incluez le fichier de connexion.

Exécutez la requête suivante :

```
SELECT id, nom, prix, stock
FROM produit
WHERE id = 1
```

Utilisez :

```
PDO::FETCH_ASSOC
```

pour obtenir un tableau associatif.

Affichez ensuite :

```
Produit : Clavier mecanique
Prix : 79.90 euros
Stock : 10
```

Question :

Quel est le type de la variable retournée par `fetch()` dans cet exercice ?

2 Exercice 2 – Créer la classe Produit

Difficulté : ⋆

On souhaite maintenant travailler avec des objets plutôt qu'avec des tableaux.

Créez une classe `Produit` contenant les attributs privés :

- `id` de type `?int` ;
- `nom` de type `string` ;
- `prix` de type `float` ;
- `stock` de type `int`.

Ajoutez :

- un constructeur ;
- les getters nécessaires ;
- une méthode `afficher()`.

La méthode `afficher()` pourra produire :

```
1 - Clavier mecanique - 79.90 euros - stock : 10
```

Créez manuellement un objet `Produit` et testez cette méthode.

3 Exercice 3 – Transformer une ligne SQL en objet

Difficulté : ⋆⋆

Reprenez la requête de l'exercice 1.

Cette fois, après le `fetch()`, créez un objet `Produit` à partir du tableau obtenu.

Vous devrez effectuer la transformation :

```
ligne SQL
    |
    v
objet Produit
```

Exemple attendu :

```
$produit = new Produit(
    ...,
    ...,
    ...,
    ...
);
```

Attention aux types retournés par PDO.

Utilisez si nécessaire :

```
(int)
(float)
```

Affichez ensuite le produit à l'aide de sa méthode `afficher()`.

Question :

Comment appelle-t-on le fait de remplir un objet avec des données provenant d'une base de données ?

4 Exercice 4 – Créer un ProduitRepository

Difficulté : ⋆⋆

On souhaite maintenant séparer :

— la représentation d'un produit ;
— l'accès à la base de données.

Créez une classe :

```
class ProduitRepository
```

Elle possède un attribut :

```
private PDO $pdo;
```

ainsi qu'un constructeur :

```
public function __construct(PDO $pdo)
```

Ajoutez ensuite la méthode :

```
public function findById(int $id): ?Produit
```

Cette méthode doit :

1. préparer une requête SQL ;
2. rechercher le produit correspondant à l'identifiant ;
3. récupérer la ligne ;
4. retourner un objet `Produit` ;
5. retourner `null` si aucun produit n'existe.

Utilisation attendue :

```
$repository = new ProduitRepository($pdo);

$produit = $repository->findById(2);

if ($produit !== null) {
    $produit->afficher();
}
```

Important :

La valeur de `$id` doit être transmise à la requête avec une **requête préparée**.

5 Exercice 5 – Hydratation et findAll()

Difficulté : ⋆⋆

Dans `ProduitRepository`, créez une méthode privée :

```
private function hydrater(array $ligne): Produit
```

Cette méthode doit transformer une ligne provenant de la base en objet `Produit`.

Modifiez ensuite `findById()` pour utiliser cette méthode.

Ajoutez maintenant :

```
public function findAll(): array
```

Cette méthode doit retourner un tableau contenant uniquement des objets `Produit`.

Utilisation :

```
$produits = $repository->findAll();

foreach ($produits as $produit) {
    $produit->afficher();
}
```

Question :

Pourquoi est-il préférable de centraliser la création des objets dans une méthode `hydrater()` ?

6 Exercice 6 – Rechercher des produits

Difficulté : ⋆⋆⋆

Ajoutez au Repository la méthode :

```
public function findByNom(string $recherche): array
```

Elle doit rechercher tous les produits dont le nom contient le texte fourni.

Exemple :

```
$produits = $repository->findByNom("clavier");
```

La requête SQL pourra utiliser :

```
LIKE
```

La valeur recherchée doit obligatoirement être transmise avec une requête préparée.

Le Repository doit retourner un tableau d'objets `Produit`.

Testez également une recherche ne retournant aucun résultat.

7 Exercice 7 – Insérer un objet

Difficulté : ⋆⋆⋆

Ajoutez la méthode suivante dans `ProduitRepository` :

```
public function insert(Produit $produit): void
```

Créez un nouvel objet :

```
$produit = new Produit(
    null,
    "Disque SSD 1 To",
    89.90,
    7
);
```

Puis enregistrez-le :

```
$repository->insert($produit);
```

La requête doit être préparée.

Question :

Pourquoi l'identifiant du nouvel objet vaut-il `null` avant l'insertion ?

Amélioration

Modifiez la classe `Produit` afin de pouvoir affecter l'identifiant généré par MySQL après l'insertion.

Vous pourrez utiliser :

```
$this->pdo->lastInsertId()
```

Après l'insertion, vérifiez que l'objet possède maintenant un identifiant.

8 Exercice 8 – Modifier un produit

Difficulté : ⋆⋆⋆

Ajoutez dans la classe `Produit` :

```
public function setNom(string $nom): void
public function setPrix(float $prix): void
public function setStock(int $stock): void
```

Ajoutez des contrôles simples :

- un nom ne doit pas être vide ;
- un prix ne peut pas être négatif ;
- un stock ne peut pas être négatif.

Ajoutez dans `ProduitRepository` :

```
public function update(Produit $produit): void
```

Testez le scénario suivant :

```
$produit = $repository->findById(1);

$produit->setPrix(74.90);
$produit->setStock(20);
```

```
$repository->update($produit);
```

Rechargez ensuite le produit depuis la base pour vérifier la modification.

Question :

Pourquoi l'instruction suivante :

```
$produit->setPrix(74.90);
```

ne modifie-t-elle pas immédiatement la base de données ?

9 Exercice 9 – Supprimer et sécuriser le CRUD

Difficulté : ⋆⋆⋆

Ajoutez la méthode :

```
public function delete(int $id): void
```

Elle doit supprimer le produit correspondant à l'identifiant.

La requête doit obligatoirement être préparée.

Testez :

```
$repository->delete(5);
```

Puis vérifiez avec :

```
$repository->findById(5);
```

Analyse

Indiquez sous forme de commentaires PHP pourquoi le code suivant est dangereux :

```
$sql = "DELETE FROM produit WHERE id = " . $_GET["id"];
```

Expliquez ensuite pourquoi une requête préparée est préférable.

10 Exercice 10 – Synthèse : catalogue de produits

Difficulté : ⋆⋆⋆⋆

Vous devez réaliser une petite application PHP orientée objet permettant de gérer un catalogue de produits.

L'application doit être organisée au minimum avec les fichiers suivants :

```
projet/
|
|-- connexion.php
|-- Produit.php
|-- ProduitRepository.php
|-- index.php
```

1. Classe Produit

La classe `Produit` doit contenir :

- `id` ;
- `nom` ;
- `prix` ;
- `stock`.

Les attributs sont privés.

Ajoutez :

- constructeur ;
- getters ;
- setters avec validation.

2. Repository

La classe `ProduitRepository` doit proposer au minimum :

```
findAll(): array

findById(int $id): ?Produit

findByNom(string $recherche): array

insert(Produit $produit): void

update(Produit $produit): void

delete(int $id): void
```

La transformation des lignes SQL en objets doit être centralisée dans une méthode :

```
private function hydrater(array $ligne): Produit
```

3. Affichage

Dans `index.php`, récupérez tous les produits et affichez-les dans un tableau HTML contenant :

- identifiant ;
- nom ;
- prix ;
- stock.

4. Recherche

Ajoutez un formulaire HTML :

```
Nom du produit : [              ]

[ Rechercher ]
```

Lorsque l'utilisateur saisit un texte, utilisez :

```
findByNom()
```

pour afficher uniquement les produits correspondants.

5. Ajout

Ajoutez un second formulaire permettant de saisir :

— nom ;
— prix ;
— stock.

À la réception du formulaire :

1. créez un objet `Produit` ;
2. transmettez-le au Repository ;
3. utilisez `insert()`.

Le code qui traite le formulaire ne doit contenir aucune requête SQL.

6. Contraintes

Votre application doit respecter les contraintes suivantes :

— aucune requête SQL dans `index.php` ;
— aucune utilisation directe de PDO dans la classe `Produit` ;
— toutes les requêtes contenant des données variables doivent être préparées ;
— le Repository retourne des objets `Produit`, et non des tableaux associatifs ;
— les valeurs affichées dans la page HTML doivent être protégées avec `htmlspecialchars()` lorsque cela est nécessaire.

7. Questions de synthèse

Répondez sous forme de commentaires dans votre code :

1. Quel est le rôle de la classe `Produit` ?
2. Quel est le rôle de `ProduitRepository` ?
3. Quel est le rôle de PDO ?
4. Qu'est-ce que l'hydratation ?
5. Quelle correspondance existe entre une ligne de la table `produit` et un objet `Produit` ?
6. Pourquoi peut-on considérer cette architecture comme une forme simple d'ORM manuel ?

Bilan

Exercice	Compétence principale
1	Lecture de données avec PDO et `FETCH_ASSOC`
2	Modélisation d'une entité métier en POO
3	Transformation d'une ligne SQL en objet
4	Création d'un Repository et méthode `findById()`
5	Hydratation et collection d'objets
6	Recherche paramétrée et requête préparée
7	CREATE : insertion d'un objet
8	UPDATE : modification d'un objet et persistance
9	DELETE et sécurité des requêtes SQL
10	Synthèse : mini-application POO + PDO + Repository

À retenir

Dans l'architecture utilisée dans ce TP :

```
Page / controleur
       |
       v
ProduitRepository
       |
       | PDO / SQL
       v
Base de donnees

       ^
       |
   Produit
```

On cherche à séparer clairement :

- les **objets métier** ;
- la **persistance des données** ;
- l'**affichage** et le traitement des requêtes utilisateur.

Ce fonctionnement permet de comprendre les mécanismes qui seront ensuite largement automatisés par des ORM comme **Eloquent** dans Laravel.